OpenSpec

A Practical Guide to Spec-Driven AI Coding

A step-by-step handbook for planning, specifying, implementing, verifying, and maintaining software changes with AI coding assistants.

Core workflow

✓ Explore the problem

✓ Propose the change

✓ Review requirements and design

✓ Implement in small tasks

✓ Verify behavior and tests

✓ Archive and sync specifications

Prepared as a practical reference. CLI commands and AI workflow syntax can vary by OpenSpec version and coding assistant; confirm them against the official documentation.

Contents

- 1. What OpenSpec is and why it helps
- 2. Install and initialize
- 3. Configure project context
- 4. Core artifacts
- 5. Commands and workflow syntax
- 6. Writing testable specifications
- 7. End-to-end workflow
- 8. Change lifecycle and folder structure
- 9. Advanced workflows
- 10. Best practices and common mistakes
- 11. Reusable AI prompts
- 12. Daily checklist
- 13. Official resources

Quick summary: OpenSpec adds a structured, reviewable layer between a feature idea and the code. It does not replace Git, automated tests, security review, or human judgment.

1. What OpenSpec is

OpenSpec is a spec-driven development workflow for working with AI coding assistants. Rather than asking an AI to implement a large feature from a single prompt, you define the intended behavior, review the plan, break the work into tasks, and then implement and verify the change.

The main benefit is control: requirements are explicit, implementation choices are reviewable, and completed work can be reflected in the project's current specifications.

The workflow at a glance

Stage	Purpose	Typical output
Explore	Clarify the problem and alternatives	Decisions and scope
Propose	Describe why the change is needed	proposal.md
Specify	Define observable behavior	Spec delta files
Design	Choose an implementation approach	design.md
Plan	Break work into manageable tasks	tasks.md
Apply	Implement the approved change	Code and updated tasks
Verify	Check requirements and tests	Test results and review
Archive	Reconcile current specs and preserve history	Updated specs and archived change

Remember: the specification describes what the system must do; the design describes how it will be built; tests provide evidence that the implementation behaves as intended.

2. Install and initialize

Prerequisites

- Node.js and npm if you choose the npm installation route.
- An existing project or a new project directory.
- A supported AI coding assistant configured with OpenSpec's workflows.
- Git is strongly recommended for safe change tracking and review.

```
node --version
npm --version
git --version
```

Install the CLI

Follow the current official installation guide. A common npm installation is:

```
npm install -g @fission-ai/openspec@latest
openspec --version
openspec --help
```

Installation requirements can change between versions. The official documentation also provides a standalone installer that may not require Node.js. Use the current supported method for your environment.

Initialize your repository

```
cd path/to/your-project
git status
openspec init
```

During initialization, select the AI coding tool you actually use. OpenSpec will create its project configuration and the workflow instructions appropriate for that tool. Restart your IDE or assistant if its instructions need to be reloaded.

Inspect the generated files

```
your-project/
■■■ openspec/
■   ■■■ config.yaml
■   ■■■ specs/
■   ■■■ changes/
■       ■■■ archive/
■■■ tool-specific workflow files/
■■■ src/
```

The exact structure varies by tool, version, and change. Treat this as an illustration, not a guaranteed file tree.

3. Configure project context

Open openspec/config.yaml and follow the configuration format supported by your installed version. Avoid copying an example configuration blindly; field names and supported options may evolve.

Useful stable project context includes:

- **Technology:** languages, frameworks, databases, APIs, and runtime versions.
- **Architecture:** folder conventions, state management, repositories, dependency injection.
- **Code conventions:** naming, error handling, logging, and formatting.
- **Testing:** unit, widget, integration, and end-to-end commands.
- **Constraints:** compatibility, security, authentication, data rules, and migration policy.
- **Boundaries:** areas that must not be changed without approval.

Keep project-wide rules in configuration or shared agent instructions. Keep feature-specific behavior in the relevant change artifacts. Do not try to document every detail of the entire application in one enormous prompt.

4. Understand the core artifacts

Artifact	Question	Typical contents
proposal.md	Why and what is in scope?	Problem, motivation, scope, capabilities, impact
specs/.../spec.md	What must the system do?	Requirements and testable scenarios
design.md	How will we build it?	Architecture, data flow, decisions, trade-offs
tasks.md	What work must be done?	Ordered, checkable implementation tasks

Behavior versus implementation

“Users can filter completed workouts by date” is a behavioral requirement. “Create a WorkoutHistoryProvider class” is an implementation decision. Keep requirements focused on observable behavior so the implementation can evolve without changing the product promise.

Current specs versus change specs

- openspec/specs/ contains the accepted, current specifications for capabilities.
- openspec/changes// contains the proposal and proposed spec deltas for work in progress.
- openspec/changes/archive/ preserves completed change records.

The folder and filenames shown in examples can vary. Use the artifacts OpenSpec actually creates for your selected workflow.

5. Commands and workflow syntax

OpenSpec has AI workflow commands and terminal CLI commands. They serve related but different purposes.

Common AI workflows

Workflow	Purpose
explore	Investigate an idea, ask questions, compare options
propose	Create the proposal and planning artifacts
apply	Implement tasks from a change
update	Revise existing planning artifacts
sync	Merge spec updates into current specs without archiving
archive	Record completed work and reconcile current specs

In an assistant that supports the /opsx: syntax, examples may look like:

```
/opsx:explore
/opsx:propose add-workout-history
/opsx:apply
/opsx:sync
/opsx:archive
```

Other tools may use different prefixes or invocation formats. Use the command form generated for your installed assistant; do not assume the examples work everywhere.

Common CLI commands

```
openspec --help
openspec list
openspec list --specs
openspec view
openspec show add-workout-history
openspec status --change add-workout-history
openspec validate add-workout-history
openspec validate --help
```

Consult the CLI help for the exact options supported by your installed version. AI workflow commands are not interchangeable with CLI commands.

A useful division of responsibility

- AI workflow: reason, draft, plan, and implement.
- CLI: inspect, validate, and manage artifacts.
- Git: branch, diff, commit, and recover changes.

6. Write good specifications

The default spec-driven format uses change operations such as ADDED, MODIFIED, REMOVED, and RENAMED. Requirements should describe observable behavior and include scenarios that can be checked.

Illustrative specification

```
## ADDED Requirements

### Requirement: View completed workout history

The system MUST allow an authenticated user
to view their completed workout records.

#### Scenario: User has completed workouts
- **WHEN** the user opens workout history
- **THEN** the system displays their completed workouts
  with exercise name, date, and score

#### Scenario: User has no completed workouts
- **WHEN** the user opens workout history
  and has no completed records
- **THEN** the system displays an empty state

#### Scenario: History cannot be loaded
- **WHEN** the system cannot retrieve the records
- **THEN** the system displays an error and allows retry
```

The four change operations

Operation	Meaning
ADDED	Introduce new requirements for a new or expanded capability.
MODIFIED	Change existing behavior; include the full updated requirement.
REMOVED	Remove a requirement and explain migration or impact when needed.
RENAMED	Rename a requirement using the required FROM and TO format.

These are specification-change operations, not Git operations. Follow the exact syntax generated by your OpenSpec version.

7. End-to-end workflow

Step 1 — Explore

Use exploration when the requirements are unclear, the codebase is unfamiliar, or there are multiple possible solutions. The aim is to understand the problem before proposing a change.

```
Explore adding workout history to my application.

Inspect the architecture, authentication, data model, and
current workout-saving flow. Identify reusable functionality,
missing requirements, edge cases, security risks, and options.

Ask me about important product decisions that cannot be
inferred safely. Do not modify application code during
exploration. Help me clarify scope before proposing the change.
```

Step 2 — Propose

Create a named change after the scope is clear.

```
/opsx:propose add-workout-history
```

Review the motivation, scope, capability names, impact, risks, and compatibility. A good proposal is small enough to explain in a few sentences.

Step 3 — Review the specification

- Can the expected behavior be described precisely?

- Are empty states, errors, and boundary cases covered?
- Are authentication and authorization explicit?
- Will existing functionality continue to work?
- Does the proposal include anything not requested?

```
Review the proposal and generated specifications against
the existing codebase and current specs. Identify missing
scenarios, ambiguity, security risks, unnecessary scope,
backward-compatibility issues, and untestable requirements.
Revise the artifacts, but do not modify application code.
List any unresolved decisions that need my approval.
```

Step 4 — Review design and tasks

Check data storage and retrieval, identity and access control, reuse of existing architecture, loading and error states, tests, and migration or indexing impacts. Break work into small, verifiable tasks.

```
## 1. Data layer
- [ ] Inspect and reuse the existing workout model.
- [ ] Implement history retrieval.
- [ ] Verify user-level access restrictions.

## 2. Application logic
- [ ] Add loading and error handling.
- [ ] Support the agreed date filter.

## 3. UI
- [ ] Display completed workout records.
- [ ] Implement empty, loading, and error states.

## 4. Verification
- [ ] Add tests for required scenarios.
- [ ] Run relevant existing tests.
- [ ] Review the complete code diff.
```

Step 5 — Apply

```
/opsx:apply
```

If the tool requires a change identifier, use the exact identifier created by the proposal. The AI should read the artifacts, implement incrementally, and update task status honestly.

```
Implement only the approved OpenSpec change. Inspect relevant
files first and follow existing conventions. Use the spec as
the behavioral contract and the design as the implementation
guide. Work incrementally. Do not replace working subsystems,
add unnecessary dependencies, expand scope, or fabricate test
results. If the plan conflicts with the codebase or needs an
unapproved design change, stop and explain the issue.
```

Step 6 — Verify

```
# Example for a Flutter project
flutter analyze
flutter test
```

Run the real tests for your stack. Add widget, integration, API, or end-to-end tests where appropriate. Report which requirements are verified by automated tests, manual inspection, not verified, or blocked.

Step 7 — Sync and archive

```
/opsx:sync
/opsx:archive
```

The exact sequence depends on the installed workflow. In the standard quickstart, archiving reconciles the spec updates and moves the completed change into the archive. Verify the final state before committing.

8. Change lifecycle and folder structure

A change named add-workout-history might contain the following artifacts before implementation:

```
openspec/
■■■ config.yaml
■■■ specs/
■   ■■■ workout-tracking/
■       ■■■ spec.md
■■■ changes/
    ■■■ add-workout-history/
        ■■■ proposal.md
        ■■■ specs/
        ■   ■■■ workout-history/
        ■       ■■■ spec.md
        ■■■ design.md
        ■■■ tasks.md
```

After completion, the change's requirements are reconciled with the main specs and the change is archived. The archive retains the proposal, design, tasks, and spec delta as a historical record.

Archive folder naming and metadata vary by version. Do not assume the date format or exact tree shown here is mandatory.

9. Advanced workflows

Workflow / feature	When it is useful
new + continue	Create and review planning artifacts one at a time.
ff	Generate all planning artifacts quickly, then review them.
verify	Run a dedicated implementation-versus-spec check if available.
bulk-archive	Archive multiple completed changes.
Custom profiles	Install only the workflows your team needs.
Custom schemas	Define different artifacts or drafting rules.
Stores / worksets	Support more advanced multi-repository planning.

These options are not necessarily installed by default. Check the documentation for the selected profile and version.

```
openspec config profile
openspec update
```

Use interactive prompts and inspect what an update changes before accepting it.

10. Best practices and common mistakes

Avoid	Prefer
A proposal to rebuild everything	Small, focused changes
Vague requirements like “make it fast”	Observable outcomes and justified targets
Coding before inspecting the codebase	Inspect architecture, tests, and existing behavior first
Treating task checkboxes as proof	Run tests and verify requirements
Silent scope expansion	Update artifacts and get approval first
Trusting AI summaries alone	Inspect the actual Git diff
Duplicate or outdated specs	Reuse capability specs and keep them current
Archiving without verification	Verify, reconcile specs, then archive

When a spec-only skip may be appropriate

Some changes, such as pure refactoring or tooling-only updates, may not change specified behavior. Use skip_specs: true only when that is genuinely appropriate and supported by your workflow. Do not use it to bypass specification work for a user-visible behavior change.

11. Reusable operating prompt

```
# OpenSpec Development Rules

Use OpenSpec as the source of truth for planning and specifying
changes in this repository.

BEFORE IMPLEMENTATION
1. Inspect the codebase, architecture, dependencies, and tests.
2. Read relevant existing OpenSpec specifications.
3. Clarify ambiguous requirements and constraints.
4. Create or update proposal, specs, design, and tasks.
5. Review scope, edge cases, security, and compatibility.
6. Do not modify application code until the plan is ready.

DURING IMPLEMENTATION
1. Follow the approved specification and design.
2. Work incrementally through tasks.
3. Reuse established components and conventions.
4. Avoid unrelated refactoring and unapproved behavior changes.
5. Stop and explain conflicts with the plan.
6. Mark tasks complete only after the work is performed.

VERIFICATION
1. Run relevant tests and static analysis.
2. Test success, failure, empty, and boundary scenarios.
3. Review authorization, validation, and data consistency.
4. Inspect the complete Git diff.
5. Report actual results, limitations, and unverified requirements.
6. Never claim a test passed unless it was executed and passed.

COMPLETION
1. Ensure implementation matches approved requirements.
2. Update artifacts when approved requirements change.
3. Reconcile specs and archive using the supported workflow.
4. Preserve relevant artifacts and code in Git.
5. Summarize behavior, changed files, tests, and limitations.
```

Treat this prompt as a reusable starting point. Adapt it to the rules and capabilities of your chosen AI coding assistant.

12. Daily change checklist

Before starting

- ■ Inspect codebase and Git status
- ■ Read related specs
- ■ Agree on scope and acceptance criteria

Before coding

- ■ Review proposal and scenarios
- ■ Review design and tasks
- ■ Resolve ambiguities and security concerns

While coding

- ■ Implement only approved scope
- ■ Track tasks honestly
- ■ Test incrementally and inspect the diff

Before finishing

- ■ Run tests and static analysis
- ■ Verify requirements and regression risks
- ■ Update and archive the change
- ■ Commit code and specs

13. Official resources

OpenSpec Quickstart — https://openspec.dev/docs/quickstart
End-to-end first-change walkthrough.

Installation guide — https://openspec.dev/docs/installation
Current installation options and prerequisites.

Spec-driven schema — https://openspec.dev/docs/schemas/spec-driven
Artifact structure and requirement format.

CLI reference — https://openspec.dev/docs/cli
Command syntax, flags, validation, and configuration.

Documentation home — https://openspec.dev/docs
Current documentation and workflow details.

Final recommendation

Master the cycle Explore → Propose → Review → Apply → Verify → Archive before customizing schemas or automating the process. Try it on two or three real features, then introduce advanced workflows as your needs grow.

This guide is a practical reference. OpenSpec evolves; confirm command syntax, CLI options, supported profiles, and installation requirements in the official documentation for your installed version.